

mscluster Field Guide

Every command worth keeping for the Qwen + LoRA Safe-RLHF work — hardware, job submission, monitoring, and the specific traps that already cost us a morning.

User

wmaboa

Login

mscluster-login

Env

conda · safe-rlhf

Compiled

25 Aug 2026

A The hardware you can reach

Node counts and specs come from the MSS Community Guidelines (Feb 2024). Note that `sinfo` reports `GRES=(null)` everywhere — this cluster does *not* tag GPUs as SLURM generic resources, so you never pass `--gres`. You pick a GPU by picking a partition.

PARTITION	NODES	GPU PER NODE	VRAM	SYSTEM RAM	MAX TIME
stampede	40	2 × GTX 1060	6 GB each · 12 GB node	32 GB	3 days
bigbatch	48	1 × RTX 3090	24 GB	128 GB	3 days
biggpu	4–7	2 × Quadro RTX 8000	48 GB each · 96 GB node	1 TB	3 days
gpuexpress	1	—	—	—	1 hour
batch	100	—	—	—	1 day

WHAT THIS MEANS FOR OUR RUNS

The frozen 7B Beaver reward model (~14 GB) plus a Qwen 1.5–3B actor and critic lands around 30–35 GB of resident weights. That fits one **biggpu** RTX 8000 comfortably. It does *not* fit bigbatch's 24 GB, and nowhere near stampede's 6 GB cards.

But house rules say biggpu is for *mature, debugged code only* — those nodes are expensive and in heavy demand. Shake the pipeline out on **bigbatch** with a small stand-in model first, then promote.

Inspect partitions and nodes

LOGIN NODE

```
# all partitions, their state, and node lists
sinfo

# partition + nodelist + generic resources
sinfo -o "%P %N %G"
```

```
# everything SLURM knows about one partition
scontrol show partition biggpu

# one node's real CPU/RAM/state
scontrol show node mscluster106
```

THE LOGIN NODE IS NOT A GPU

Running `nvidia-smi` after you SSH in shows an RTX 2060 SUPER with 8 GB. That card belongs to the login node and is for smoke-testing only. To see a real training GPU you must be inside a job on a worker node.

B First-time setup, in order

This sequence matters — each step fails in a confusing way if the one before it was skipped.

01 Install your own Miniconda

MSS explicitly recommends a personal install over the shared one. Lands in `~/miniconda3`.

One-time install

LOGIN NODE

```
cd ~
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda_install.sh
bash miniconda_install.sh -b -p $HOME/miniconda3
$HOME/miniconda3/bin/conda init bash
source ~/.bashrc

which conda      # must return a path before continuing
conda --version
```

02 Accept the channel Terms of Service

Modern conda refuses any *non-interactive* env creation until these are accepted. Skip this and every `sbatch` env build dies instantly with `CondaToSNonInteractiveError`.

One-time, account-level

LOGIN NODE

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkg/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkg/r
```

03 Get the code on the cluster

A clone is light enough to run on the login node. Let it finish *before* submitting any job that reads the directory.

Clone the framework

LOGIN NODE

```
git clone https://github.com/PKU-Alignment/safe-rlhf.git ~/safe-rlhf
```

04 Build the environment as a job

Conda's solver is heavy enough that MSS asks you to keep it off the login node.

env_setup.sbatch

WRITE TO FILE

```
#!/bin/bash
#SBATCH --partition=bigbatch
#SBATCH --nodes=1
#SBATCH --time=01:00:00
#SBATCH --output=env_setup_%j.out

set -euo pipefail  # fail loudly, never cascade into base

source "$HOME/miniconda3/etc/profile.d/conda.sh"

cd ~/safe-rlhf
CONDA_OVERRIDE_CUDA=11.8 conda env create --file conda-recipe.yaml
conda activate safe-rlhf
echo "active env: $CONDA_DEFAULT_ENV"
pip install peft

python -c "import torch, transformers, deepspeed, peft; \
print(torch.__version__, torch.cuda.is_available(), torch.cuda.device_count())"
```

05 Pin the two versions that break things

Upstream leaves `transformers` unbounded, so the solver grabs 5.x — which removed the `tokenization_utils` paths `safe-rlhf` imports. And `MKL ≥ 2024.1` dropped the `itnotify` symbols PyTorch was built against.

Run inside the activated env

SBATCH JOB

```
conda install -n safe-rlhf -y "mkl=2024.0.0" "transformers>=4.37.2,<4.47"
```

c The job script pattern

Every job you submit should carry these four habits: source conda by absolute path, `set -euo pipefail`, echo the active env, and write output somewhere named after the job ID.

Skeleton to copy for any run

WRITE TO FILE

```
#!/bin/bash
#SBATCH --job-name=rm-smoke
#SBATCH --partition=bigbatch
#SBATCH --nodes=1
#SBATCH --time=02:00:00
#SBATCH --output=logs/rm_smoke_%j.out

set -euo pipefail

# sbatch runs a NON-interactive shell — .bashrc's conda block is skipped
source "$HOME/miniconda3/etc/profile.d/conda.sh"
conda activate safe-rlhf

echo "node : $(hostname)"
echo "env   : $CONDA_DEFAULT_ENV"
nvidia-smi --query-gpu=name,memory.total --format=csv

cd ~/safe-rlhf
# ... your training command here ...
```

Useful `#SBATCH` flags

- `-w mscluster61` — demand one specific node.
- `--nodelist=mscluster[2-4]` — a specific set.
- `--nodes=2` — count only, scheduler picks.
- `%j` in `--output` — expands to the job ID, so runs never overwrite each other.

Prefer `sbatch` over `srun`

MSS advises against interactive `srun`: it ties up your terminal, and load shedding kills interactive sessions badly. Submit and walk away instead.

Submit, watch, finish

The everyday loop

[LOGIN NODE](#)

```
# submit — prints the JOBID you'll need below
sbatch my_job.sbatch

# is it queued / running? (note: no $ on the username)
queue -u wmaboa

# the output file is written live — safe to read mid-run
cat logs/rm_smoke_<JOBID>.out
tail -f logs/rm_smoke_<JOBID>.out

# once it's gone from queue: did it succeed or fail?
sacct -j <JOBID> --format=JobID,JobName,State,ExitCode,Elapsed

# stop things
scancel <JOBID>
scancel --user=wmaboa
```

A job disappears from `squeue` the moment it finishes — success and failure look identical there. `sacct` is the one that tells you which happened.

Reading the `ST` and reason columns

CODE	MEANS
<code>R</code>	Running right now.
<code>PD</code>	Pending — waiting in the queue. The reason column says why.
<code>CG</code>	Completing — finishing up and releasing resources.
<code>Priority</code>	Nothing is wrong; other jobs are simply ranked ahead of yours.
<code>Resources</code>	You're next in line, but the nodes you asked for are still busy.
<code>Dependency</code>	Waiting on another job you linked with <code>--dependency</code> .
<code>ReqNodeNotAvail</code>	A node you named with <code>-w</code> is down or draining. Pick another.

Node states in `sinfo` read the same way: `idle` free, `alloc` fully in use, `down*` unreachable, `inval` in an invalid state.

E Traps we already hit

Each of these cost real time on 18 August. Symptom on top, fix underneath.

```
slurm_script: line 7: conda: command not found
```

sbatch runs a non-interactive shell, which skips the conda block in `.bashrc`. Always `source "$HOME/miniconda3/etc/profile.d/conda.sh"` by absolute path at the top of every job script.

```
CondaToNonInteractiveError: Terms of Service have not been accepted
```

Run the two `conda tos accept` commands from step B-2. One time, then it stops.

```
error: externally-managed-environment
```

You're hitting the node's **system Python**, not your env — meaning activation silently failed earlier in the script. Don't reach for `--break-system-packages`; fix the activation. `set -euo pipefail` makes this fail at the real cause instead of cascading.

8/25/26, 7:36 AM

mscluster Field Guide

ImportError: libtorch_cpu.so: undefined symbol: iJIT_NotifyEvent

MKL ≥ 2024.1 removed symbols PyTorch expects. `conda install -n safe-rlhf "mkl=2024.0.0"`.

ImportError: cannot import name 'PaddingStrategy' from 'transformers.tokenization_utils'

The solver installed `transformers` 5.x. safe-rlhf needs 4.x — pin `"transformers>=4.37.2,<4.47"`.

This pin is deliberate and must be documented, not a leftover.

Job started before `git clone` finished → cd into a half-populated directory

Submitting while a clone is still running is a race. Confirm `ls ~/safe-rlhf` looks complete before `sbatch`.

Old errors reappearing in a log you just checked

`cat env_setup_*.out` concatenates *every* past run. Read one job at a time —

`cat env_setup_<JOBID>.out` — and delete logs you're done with.

F

Where to put things

PATH	USE IT FOR	SIZE / LIFETIME
/home-mscluster/wmaboa/	Code, conda envs, day-to-day working space. Where you land on login.	Up to 50 GB · permanent
/scratch/wmaboa/	Heavy disk-IO during a run. Local to each node, so staged per node.	A few weeks
/gluster/wmaboa/	Fast network scratch — same view from every node, no per-node staging.	A few months
/datasets/wmaboa/	Model checkpoints and corpora over 50 GB.	50 GB → terabytes

DOWNLOADING LARGE FILES

Fetch straight onto the storage host rather than pulling through your home directory:

`ssh wmaboa@10.100.14.250` for `/datasets`, `10.100.14.2` for `/gluster`. HuggingFace checkpoints for the Beaver reward model belong in `/datasets`, with `HF_HOME` pointed there so the cache doesn't quietly eat your 50 GB home quota.

G

House rules

Escalate in the right order

Never email staff directly. One ticket per issue to `support@wits-mss.supportsystem.com`, subject line **TWK HPC Query**, from your student address. Then MSS meeting → supervisor → Head of School.

Climb the ladder of hardware

stampede first, bigbatch if that's not enough, biggpu only when nothing else fits *and* the code is debugged.

Leave the login node alone

No heavy CPU, memory, or disk work outside `sbatch`. Everyone shares it.

Exit cleanly

Ghost processes outlive badly-terminated jobs and hold nodes hostage. Make sure training scripts exit properly.

Plan around load shedding

The UPS survives roughly one event per day. Checkpoint often; long unattended runs are a gamble during heavy shedding.

September–November is full

Historically near-zero headroom in those months. Get the expensive runs done before then.

Required acknowledgement for the thesis and any publication using these results:

“Computations were performed using High Performance Computing infrastructure provided by the Mathematical Sciences Support unit at the University of the Witwatersrand, Johannesburg.”

Sources: MSS Community Guidelines, 8 February 2024 · live `sinfo`/`scontrol` output, 18 August 2026 · PKU-Alignment safe-rlhf.

